Documentação - Projeto Integrador CPR

Descrição do Problema

O gerenciamento financeiro realizado de forma manual ou por meio de sistemas legados engessados (como a antiga versão em Delphi 4) dificulta muito o acompanhamento de contas a pagar e a receber. Essa falta de controle moderno e integrado acaba gerando atrasos, perda de prazos de descontos e falta de visão real da saúde financeira da empresa.

Objetivos da Aplicação

Desenvolver e modernizar o sistema financeiro através de uma API REST baseada em Java e Spring Boot. O objetivo é automatizar o controle de obrigações e direitos de forma eficiente, criando uma base sólida e independente de frontend (arquitetura monolítica moderna) que facilite manutenções, escalabilidade e a futura migração para microsserviços.

Descrição das Regras de Negócio

- Uma conta já paga não pode ser paga novamente, e uma conta recebida não pode ser recebida duas vezes (imutabilidade de título quitado).
- O desconto de quitação/recebimento só é aplicado se a data de pagamento informada for menor ou igual à data de validade do desconto.
- Não é possível excluir títulos financeiros que já estejam com status de quitado (pago ou recebido).
- Títulos só podem ser lançados associados a Pessoas (clientes ou fornecedores) e Planos de Contas que existam no banco de dados.

Descrição dos Requisitos Funcionais

- Cadastro, consulta, alteração e exclusão de Pessoas (classificadas em Clientes, Fornecedores ou Ambos).
- Cadastro, consulta, alteração e exclusão de categorias no Plano de Contas.
- Lançamento, edição, listagem (com filtros por data e situação) e exclusão de Contas a Pagar e Contas a Receber.
- Quitação de despesas e recebimento de receitas com cálculo e aplicação automática de descontos e acréscimos.

Descrição dos Requisitos Não-Funcionais

- Aplicação deve ser desenvolvida em Java (versão 21) usando o framework Spring Boot 3.
- Utilização de banco de dados relacional embutido SQLite para facilitar o ambiente local.

- A aplicação deve ser estruturada como uma API REST, comunicando-se via JSON.
- Documentação dos endpoints obrigatória via Swagger/OpenAPI.
- Validação estrita dos dados de entrada (payload) para evitar falhas ou inconsistências no banco.

Descrição das Principais Funcionalidades

- **Gerenciamento de entidades:** CRUD de Pessoas e Planos de Contas.
- **Lançamento financeiro:** Criação de títulos de Contas a Pagar e Contas a Receber, informando tipo do documento, datas, valores e envolvidos.
- **Fluxo de Liquidação:** Operações específicas de pagamento e recebimento que alteram o status das contas para "PAGA" ou "RECEBIDA".
- **Buscas:** Listagem flexível e filtrada de pendências financeiras.

Modelo de Domínio

As entidades baseiam-se em Pessoas (que englobam tanto Clientes quanto Fornecedores), PlanoConta (uma árvore de categorias financeiras), ContaPagar e ContaReceber. Elas se relacionam diretamente entre si, onde cada transação precisa obrigatoriamente de uma pessoa e uma categoria.

Diagrama	Arquivo / Referência
Modelo de Domínio	modelo de domínio.png
Diagrama de Classes	Diagrama de Classes.png
DER do Banco de Dados	DER do Banco de Dados.png

Arquitetura Monolítica

O sistema foi estruturado em camadas de fácil compreensão, com isolamento das responsabilidades em rotas HTTP, lógicas de transação, e banco de dados.

Diagrama de Arquitetura	Arquivo / Referência
Arquitetura Monolítica	Arquitetura Monolitica.jpeg
Diagrama de Componentes	Diagrama de Componentes.jpeg
Diagrama de Pacotes	Diagrama de Pacotes.png
Diagrama de Casos de Uso	Diagrama de Casos de Uso.png

Diagrama de Arquitetura	Arquivo / Referência
Diagrama de Sequência	Diagrama de Sequencia.jpeg

Descrição das APIs

A aplicação expõe uma interface de comunicação RESTful organizada por domínios (/api/pessoas, /api/plano-contas, /api/contas-pagar, /api/contas-receber). Aceita e devolve cargas no formato JSON. O desenho utiliza métodos HTTP semanticamente corretos: POST para cadastro, GET para listagens e buscas, PUT para atualizações e DELETE para exclusão de registros.

Estratégia de Persistência

Adotou-se uma abordagem relacional através do banco de dados SQLite, gerenciado diretamente pelo Spring Data JPA e Hibernate. As entidades Java são mapeadas para as tabelas do banco, garantindo o gerenciamento transparente das transações sem a necessidade de escrever queries em SQL puro.

Tratamento de Falhas

O projeto possui um mecanismo global e unificado de intercepção de exceções (ApiExceptionHandler). Erros de regras de negócio estouram e retornam com status 422 (Unprocessable Entity). Erros em que recursos não são achados retornam 404 (Not Found). Erros de preenchimento ou formato JSON (campos vazios ou malformados) são barrados e formatados de forma amigável com status 400 (Bad Request).

Procedimentos para Executar a Aplicação

1. Certifique-se de ter a JDK 21 instalada na máquina.
2. Baixe os arquivos fontes ou clone o repositório do projeto.
3. Via terminal, acesse a pasta raiz da aplicação (onde está o arquivo `pom.xml`).
4. Execute o comando `mvn spring-boot:run`. O sistema cuidará de inicializar o banco de dados e subir o servidor na porta padrão local.

Testes Realizados

Baseado na etapa 1, os testes de integração do Swagger cobriram:

- Cadastro de clientes (Pessoa Física) e fornecedores (Pessoa Jurídica).
- Cadastro de Planos de Contas para categorias PIX e Cartão, e bloqueios para código duplicado.
- Criação de contas a pagar e receber (Fluxo feliz associando IDs corretos).

- Operações de quitação e recebimento com sucesso, aplicando ou não descontos no prazo.
- Regras de negócio de falha: tentar quitar com desconto fora da validade e tentar quitar uma conta que já está paga/recebida.
- Tratamento de relacionamentos inexistentes: lançar contas com pessoas e planos de contas que não existem.
- Tolerância da API contra falhas (idempotência): tentar fazer a chamada de pagamento repetidamente, IDs inexistentes e recebimento de Enum/Payload corrompidos ou inválidos.
- Verificação de saúde da aplicação via Actuator (Health Check).

Dificuldades Encontradas

As principais dificuldades giraram em torno da configuração e adaptação do banco de dados SQLite. Fazer com que o dialeto do SQLite conversasse perfeitamente e gerasse as tabelas com o Hibernate na versão mais nova do Spring Boot deu certo trabalho. Além disso, a conversão e o parsing das strings de datas enviadas pelos payloads JSON (`LocalDate`) renderam algumas dores de cabeça até acharmos a padronização ideal. Unificar clientes e fornecedores na mesma tabela Pessoa, ao invés de usar tabelas separadas como o Delphi fazia, também exigiu criar bons mecanismos de validação para manter a integridade visualmente simples, mas complexa por trás.